TERRAFORM TRAINING SERIES

Terraform Modules

Complete Guide — From Scratch to Production

Author
Aviz & Claude

Cloud Provider
Amazon Web Services (AWS)

Series
Terraform Zero to Hero

Year
2026

1. What is a Module?

A Terraform module is a collection of .tf files grouped inside a directory that together define a set of reusable infrastructure resources.

Every Terraform configuration you write is technically a module — even a single directory with one main.tf is called the root module. When you create sub-directories and call them from the root, those are called child modules.

Core Concept:

```
module  =  directory  +  .tf files inside it
```

A module accepts inputs (via variables), creates resources,
and can return outputs to whoever called it.

2. Why Use Modules?

Without modules, as your infrastructure grows, you end up copying and pasting the same resource blocks across multiple .tf files or workspaces. This causes:

- Code duplication — same EC2 block repeated for dev, uat, prd
- Inconsistency — one environment has a setting another does not
- High maintenance overhead — change in one place needs to be updated everywhere

Modules solve this:

Problem (Without Modules)	Solution (With Modules)
Same EC2 block copy-pasted 3 times	Define EC2 once in a module, call it 3 times
Change instance type in 5 files	Change it in one module
Hard to share infra patterns across teams	Package module and share via Git or Terraform Registry
No clear separation of concerns	Each module owns one resource type

3. Types of Modules

Root Module

- The working directory from which you run terraform init / terraform apply
- Every project has exactly one root module
- Contains the provider block and calls child modules

Child Module

- A sub-directory that the root module (or another module) calls using a module block
- Does NOT contain a provider block — inherits from root
- Has its own variables.tf, main.tf, and outputs.tf

Published / Remote Module

- Hosted on the Terraform Registry, GitHub, S3, or other remote sources
- **Called with a remote `source` URL instead of a local path**
- Example: `source = "terraform-aws-modules/ec2-instance/aws"`

4. Module Structure — File Layout

This is the recommended project layout when using local child modules:

```
project/                        <-- Root Module
│
├── main.tf                     # Provider block + module calls
├── variables.tf                # Root-level input variables
├── terraform.tfvars            # Actual values for root variables
│
└── modules/                    # All child modules live here
    │
    ├── ec2/                    <-- EC2 Child Module
    │   ├── main.tf             # Resource: aws_instance
    │   ├── variables.tf        # Inputs: ami, instance_type, name
    │   └── outputs.tf          # Outputs: instance_id, public_ip, private_ip
    │
    └── s3/                     <-- S3 Child Module
        ├── main.tf             # Resource: aws_s3_bucket
        ├── variables.tf        # Input: bucket_name
        └── outputs.tf          # Outputs: bucket_id, bucket_arn
```

Why these three files per module?

File	Purpose
variables.tf	Defines what inputs the module accepts
main.tf	Contains the actual resource definitions
outputs.tf	Defines what data the module exposes back to the caller

5. Building a Module — Step by Step

5.1 EC2 Module

Step 1 — Create the directory

```
mkdir -p modules/ec2
```

Step 2 — Define inputs (modules/ec2/variables.tf)

This file declares what the caller must (or can optionally) provide to this module.

```
variable "ami" {
  description = "AMI id to use for the EC2 instance"
  type        = string
  # No default — caller MUST provide this value
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"   # Optional — caller can override
}

variable "name" {
  description = "Name tag for the EC2 instance"
  type        = string
  default     = "ec2-instance"
}
```

Key Points:
- Variables WITH default → optional inputs (caller may skip them)
- Variables WITHOUT default → required inputs (caller must supply them)
- type enforces what kind of value is accepted: string, number, bool, list, map, etc.

Step 3 — Define the resource (modules/ec2/main.tf)

```
resource "aws_instance" "instance" {
  ami           = var.ami            # Value from modules/ec2/variables.tf
  instance_type = var.instance_type  # Value from modules/ec2/variables.tf

  tags = {
    Name = var.name                  # Value from modules/ec2/variables.tf
  }
}
```

Inside a module, reference your own variables using var.<variable_name>. The module has no idea where those values come from — it just consumes what it receives.

Step 4 — Define outputs (modules/ec2/outputs.tf)

Outputs make specific attributes of the created resource accessible to whoever called this module.

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.instance.id
}

output "public_ip" {
  description = "Public IP of the EC2 instance"
  value       = aws_instance.instance.public_ip
}

output "private_ip" {
  description = "Private IP of the EC2 instance"
  value       = aws_instance.instance.private_ip
}
```

5.2 S3 Module

Step 1 — Create the directory

```
mkdir -p modules/s3
```

Step 2 — Define inputs (modules/s3/variables.tf)

```
variable "bucket_name" {
  description = "Name of the S3 bucket to create"
  type        = string
  # Required — no default, caller must provide a globally unique name
}
```

Step 3 — Define the resource (modules/s3/main.tf)

```
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name

  tags = {
    Name = var.bucket_name
  }
```

```
}
```

The resource label "this" is a widely used convention when a module manages exactly one resource of a given type. It keeps references clean: aws_s3_bucket.this.id

Step 4 — Define outputs (modules/s3/outputs.tf)

```
output "bucket_id" {
  description = "ID of the S3 bucket"
  value       = aws_s3_bucket.this.id
}

output "bucket_arn" {
  description = "ARN of the S3 bucket"
  value       = aws_s3_bucket.this.arn
}
```

6. Root Module — Calling the Modules

The root module ties everything together. It defines the provider, declares root-level variables, calls child modules, and optionally surfaces module outputs.

Root variables.tf (project/variables.tf)

```
variable "region" {
  description = "AWS region for provider"
  type        = string
  default     = "ap-south-1"
}

variable "ec2_ami" {
  description = "AMI to use for the EC2 instance"
  type        = string
  # Required — no default
}

variable "ec2_instance_type" {
  type    = string
  default = "t3.micro"
}

variable "ec2_name" {
  type    = string
  default = "dev-ec2"
}

variable "s3_bucket_name" {
  description = "Name for the S3 bucket (must be globally unique)"
  type        = string
  # Required — no default
}
```

Root main.tf (project/main.tf)

```
# Provider block — only in the root module
provider "aws" {
  region = var.region
}

# Calling the EC2 module
module "ec2_instance" {
  source        = "./modules/ec2"         # Path to EC2 child module
  ami           = var.ec2_ami              # Root variable → module input
  instance_type = var.ec2_instance_type    # Root variable → module input
  name          = var.ec2_name             # Root variable → module input
}
```

```
# Calling the S3 module
module "s3_bucket" {
  source      = "./modules/s3"            # Path to S3 child module
  bucket_name = var.s3_bucket_name         # Root variable → module input
}

# Surfacing module outputs at root level
output "ec2_instance_id" {
  value = module.ec2_instance.instance_id  # Access EC2 module output
}

output "ec2_public_ip" {
  value = module.ec2_instance.public_ip
}

output "s3_bucket_id" {
  value = module.s3_bucket.bucket_id
}
```

7. How Data Flows: Input → Module → Output

Understanding the data flow is critical. Here is the complete flow from tfvars to the created resource and back:

```
terraform.tfvars
      |  ec2_ami = "ami-03695d52f0d883f65"
      v
root variables.tf
      |  variable "ec2_ami" { type = string }
      v
root main.tf  (module block)
      |  module "ec2_instance" {
      |    ami = var.ec2_ami   ──────────────────────┐
      |  }                                            |
      v                                               |
modules/ec2/variables.tf                              |
      |  variable "ami" { type = string }  <──────────┘
      v
modules/ec2/main.tf
      |  resource "aws_instance" "instance" {
      |    ami = var.ami
      |  }
      v
AWS EC2 Instance is created
      |  instance.id, instance.public_ip, instance.private_ip
      v
modules/ec2/outputs.tf
      |  output "instance_id" { value = aws_instance.instance.id }
      v
root main.tf accesses:
      |  module.ec2_instance.instance_id
      v
Displayed in terminal after terraform apply
```

Summary of access patterns:

Context	How to Reference
Inside a module — its own variable	var.variable_name
Inside a module — its own resource attribute	resource_type.resource_label.attribute
Root module accessing a child module output	module.module_label.output_name
Root module accessing its own variable	var.variable_name

8. The module Block — Syntax Explained

```
module "<module_label>" {
  source = "<path_or_url>"

  # Input arguments — must match variable names in the module's variables.tf
  <variable_name_1> = <value>
  <variable_name_2> = <value>
}
```

Component	Meaning
module_label	Unique name for this call — used to reference outputs as module.<label>.<output>
source	Path to the module directory (local) or URL (remote)
Input arguments	Key-value pairs matching variable names in the child module's variables.tf

Source path examples:

```
# Local paths
source = "./modules/ec2"            # relative path from root
source = "../shared/networking"      # parent directory

# Terraform Registry
source  = "terraform-aws-modules/ec2-instance/aws"
version = "~> 5.0"

# GitHub
source = "git::https://github.com/myorg/tf-modules.git//modules/ec2?ref=v1.2.0"
```

9. Module Outputs — Passing Data Back

Child modules do not automatically expose all resource attributes. You must explicitly declare what you want to share via outputs.tf inside the module.

Inside the module (modules/ec2/outputs.tf):

```
output "instance_id" {
  value = aws_instance.instance.id
}
```

In the root module (project/main.tf):

```
output "ec2_instance_id" {
  value = module.ec2_instance.instance_id
}
```

The pattern is always: module.<module_label>.<output_name_defined_in_child_outputs_tf>

Passing one module's output as input to another:

```
module "ec2_instance" {
  source        = "./modules/ec2"
  ami           = var.ec2_ami
  instance_type = var.ec2_instance_type
  name          = var.ec2_name
}

module "monitoring" {
  source      = "./modules/monitoring"
  instance_id = module.ec2_instance.instance_id  # EC2 output → monitoring input
}
```

10. Using terraform.tfvars with Modules

The terraform.tfvars file only supplies values to the root module's variables. You do not pass values directly to child module variables from tfvars — that happens through the module block in main.tf.

project/terraform.tfvars

```
region             = "ap-south-1"
ec2_ami            = "ami-03695d52f0d883f65"
ec2_instance_type  = "t3.micro"
ec2_name           = "dev-ec2"
s3_bucket_name     = "my-test-bucket-12345-unique"   # Must be globally unique
```

Per-environment tfvars files

Maintain separate tfvars files per environment and apply them selectively:

```
# dev.tfvars
ec2_instance_type  = "t3.micro"
ec2_name           = "dev-ec2"
s3_bucket_name     = "myapp-dev-state-bucket"

# prd.tfvars
ec2_instance_type  = "t3.large"
ec2_name           = "prd-ec2"
s3_bucket_name     = "myapp-prd-state-bucket"
```

```
# Apply per environment:
terraform apply -var-file="dev.tfvars"
terraform apply -var-file="prd.tfvars"
```

11. Initializing and Running — Commands

Step 1 — Navigate to root module

```
cd project
```

Step 2 — Initialize

```
terraform init
```

What happens during init with modules:

- **Local modules:** Terraform resolves the path
- **Remote modules:** Downloads into .terraform/modules/

Re-run terraform init every time you:
• Add a new module block
• Change the source of an existing module
• Update the version of a remote module

Step 3 — Validate

```
terraform validate
```

Step 4 — Preview

```
terraform plan -var-file="terraform.tfvars"
```

The plan output shows resources grouped under their module path. Example:

```
  # module.ec2_instance.aws_instance.instance will be created
  # module.s3_bucket.aws_s3_bucket.this will be created
```

Step 5 — Apply

```
terraform apply -var-file="terraform.tfvars"

# With auto-approval (use carefully):
terraform apply -var-file="terraform.tfvars" -auto-approve
```

Step 6 — View Outputs

```
terraform output

# Sample output:
ec2_instance_id = "i-0abc123def456"
ec2_public_ip   = "13.232.xx.xx"
s3_bucket_id    = "my-test-bucket-12345-unique"
```

Step 7 — Destroy

```
# Destroy everything:
terraform destroy -var-file="terraform.tfvars"

# Destroy only a specific module:
terraform destroy -target=module.ec2_instance -var-file="terraform.tfvars"
```

12. State File Behavior with Modules

When modules are used, Terraform tracks resources in the state file with their full module path as a prefix:

```
module.<module_label>.<resource_type>.<resource_label>

# Examples:
module.ec2_instance.aws_instance.instance
module.s3_bucket.aws_s3_bucket.this
```

Useful state commands with modules:

```
# List all resources — module resources appear with their module path
terraform state list

# Output:
# module.ec2_instance.aws_instance.instance
# module.s3_bucket.aws_s3_bucket.this

# Inspect a specific module resource
terraform state show module.ec2_instance.aws_instance.instance

# Replace (taint) a module resource
terraform plan -replace="module.ec2_instance.aws_instance.instance"
```

WARNING — Module label renaming:
If you rename a module block label (e.g. ec2_instance → ec2_server),
Terraform sees it as a destroy + create because the state path changed.
Use terraform state mv to rename the state entry instead:

```
terraform state mv \
  module.ec2_instance.aws_instance.instance \
  module.ec2_server.aws_instance.instance
```

13. Module Versioning & Remote Sources

When using modules from the Terraform Registry or GitHub, always pin a version to avoid unexpected breaking changes.

Terraform Registry (public modules)

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"   # Allows 5.x but not 6.x

  name = "my-vpc"
  cidr = "10.0.0.0/16"
  azs  = ["ap-south-1a", "ap-south-1b"]
}
```

Version constraint operators:

Operator	Meaning
= 5.1.0	Exactly 5.1.0
>= 5.0.0	5.0.0 or higher
~> 5.0	5.x only (minor + patch updates)
~> 5.1.0	5.1.x only (patch updates)

GitHub source

```
module "ec2" {
  source = "git::https://github.com/myorg/tf-modules.git//modules/ec2?ref=v1.2.0"
}
```

14. Common Mistakes and How to Avoid Them

Mistake 1 — Forgetting to run terraform init after adding a module

Symptom:	Error: Module not installed or Could not load plugin
Fix:	Always run terraform init after adding or changing a module block

Mistake 2 — Putting provider block inside a child module

Child modules should NOT define provider blocks. The root module owns the provider.

```
# WRONG — in modules/ec2/main.tf:
provider "aws" {
  region = "ap-south-1"
}

# CORRECT — only in project/main.tf (root module)
```

Mistake 3 — Not passing a required variable to the module

Symptom:	Error: Missing required argument
Fix:	Check variables.tf for variables without default and ensure you pass them in the module block

Mistake 4 — Referencing an output not defined in the module

Symptom:	Error: Unsupported attribute — module does not have attribute xyz
Fix:	Add the output to the module's outputs.tf then re-run terraform apply

Mistake 5 — Renaming a module label without updating state

Symptom:	Terraform plans to destroy and recreate the resource even though nothing changed
Fix:	Use terraform state mv to rename the state entry to match the new module label

Mistake 6 — Using count or for_each on a module call

When you use count on a module call, Terraform creates multiple instances of ALL resources defined in that module.

```
module "ec2_instance" {
  count         = 3
  source        = "./modules/ec2"
  ami           = var.ec2_ami
  instance_type = var.ec2_instance_type
  name          = "instance-${count.index}"
}

# Access individual instance outputs:
module.ec2_instance[0].instance_id
module.ec2_instance[1].instance_id
module.ec2_instance[2].instance_id
```

15. Best Practices Checklist

- ☐ One module per resource type — keep EC2, S3, VPC as separate modules
- ☐ Always define description on every variable and output — it serves as documentation
- ☐ Always add type to every variable to catch incorrect input early
- ☐ Use default values for optional inputs; leave required inputs without default
- ☐ Never put provider block inside a child module
- ☐ Always pin version when using remote modules from Registry or GitHub
- ☐ Re-run terraform init every time you add or change a module block
- ☐ Use terraform state mv if you rename a module label to avoid resource recreation
- ☐ Expose only needed attributes via outputs.tf — mark sensitive values with sensitive = true
- ☐ Keep modules small and focused — a module that does too much becomes hard to reuse
- ☐ Store modules in Git with semantic version tags for controlled rollouts
- ☐ Validate before apply — always run terraform validate and terraform plan first

Quick Reference — Command Cheatsheet

```
# Initialize — MUST run after adding/changing any module block
terraform init

# Validate syntax and variable references
terraform validate

# Preview
terraform plan -var-file="terraform.tfvars"

# Apply
terraform apply -var-file="terraform.tfvars"
terraform apply -var-file="terraform.tfvars" -auto-approve

# List all resources including module resources
terraform state list

# Inspect a module resource
terraform state show module.ec2_instance.aws_instance.instance

# Destroy a specific module
terraform destroy -target=module.ec2_instance -var-file="terraform.tfvars"

# Rename a module in state (after renaming the module label in main.tf)
terraform state mv \
  module.old_label.aws_instance.instance \
  module.new_label.aws_instance.instance

# View all outputs
terraform output

# View a specific output
terraform output ec2_instance_id
```